

EXTERNAL AI ASSISTANT & BLUETOOTH MIC CAPTURE

DATE: Oct 2, 2026
BRANCH: revive2
BUILDS: #2126100204 -
#2126100211

[EXECUTIVE OVERVIEW]

PRODUCTION VERIFIED

Today’s engineering upgrades transformed Arcane from a passive launcher/assistant router into an autonomous, hardware-aware voice bridge. When an operator triggers voice commands from a connected smartwatch, earbuds, or Bluetooth steering wheel, Arcane intercepts the hardware intent, unlocks the keyguard if needed, routes bidirectional audio to the wearable via **Bluetooth SCO**, launches external frontier LLMs (ChatGPT, Gemini, Claude, Perplexity, Copilot, or custom packages), and auto-clicks their internal microphone using precision calibrated touch gestures after an audio-channel sync delay.

1. COMPREHENSIVE LOG OF ASSISTANT CHANGES MADE TODAY

All core assistant features developed across git commits and working-tree improvements today (Oct 2, 2026):

COMMIT / BUILD	FEATURE SCOPE	ENGINEERING DETAILS & IMPACT
Working Tree UNCOMMITTED	Active Recording Guard & SCO Watchdog	• Audio Session Protection (<code>sessionLikelyActive</code>): Evaluates <code>am.activeRecordingConfigurations</code> . If external AI holds the mic, repeat watch button clicks skip auto-tap to prevent toggling voice OFF. • SCO Watchdog Extended: Bumped timeout from 60s to 120s. • Nora Voice Retrigger Hook: Prevents stacking duplicate screens on repeated assistant clicks. • Debug Broadcast Receiver: <code>me.ihjas.missions.DEBUG_SCO</code> for ADB validation.
c3006fc BUILD 2126100211	Configurable Mic Tap Delay & SCO Audio Mode	• Mic Tap Latency Calibration: Adds 200ms–4000ms delay slider & quick chips (500, 800, 1000, 1200, 1500, 2000, 3000ms) to allow Bluetooth SCO handshakes to settle. • Bluetooth SCO Call Simulation (<code>BluetoothAudioRouter.kt</code>): Simulates <code>MODE_IN_COMMUNICATION</code> with low-level silent PCM keep-alive track for call-only smartwatches.
b17f0b4 BUILD 2126100209	Automated Lock Screen Unlock Sequence	• Two-Step Sequence: Movement 1 (Screen unlock swipe) → Movement 2 (Launch assistant + mic tap). • <code>UnlockSensorOverlay.kt</code> : Mounts over Keyguard upon screen wake, intercepts operator swipe-up, and commits on <code>ACTION_USER_PRESENT</code> . • Added unlock calibration and test actions to Settings.

COMMIT / BUILD	FEATURE SCOPE	ENGINEERING DETAILS & IMPACT
a38ce31 BUILD 2126100204	Multi-Method Voice Tap Calibration	4 calibration modes: Reticle (crosshair overlay), Touch Sensor (raw tap tracker), Auto-Detect (accessibility node drilldown), and Manual Coords. - Upgraded click replay to <code>dispatchGesture()</code> physical touch coordinates.

2. DEEP-DIVE: HOW BLUETOOTH MIC AUTO-CAPTURE WORKS

The primary hurdle in external assistant integration is that Android treats third-party apps as isolated sandboxes. When launched via Bluetooth triggers, apps like ChatGPT or Gemini default to the phone's internal microphone rather than the smartwatch/headset. Arcane resolves this through a multi-tier audio stack:

A. Audio Routing & Bluetooth SCO Channel (`BluetoothAudioRouter.kt`)

- 1. Communication Mode Activation:** `audioManager.mode = AudioManager.MODE_IN_COMMUNICATION` .
This instructs Android's AudioFlinger to prioritize two-way voice communication over standard A2DP media playback.
- 2. Android 12+ Communication Device Binding:** On Android 12 (API 31+), queries `availableCommunicationDevices` and explicitly selects `TYPE_BLUETOOTH_SCO` , `TYPE_BLE_HEADSET` , or `TYPE_HEARING_AID` via `audioManager.setCommunicationDevice(btDevice)` .
- 3. Forced Bluetooth SCO Call Simulation (For Call-Only Watches):** Many smartwatches (especially WearOS and proprietary fitness watches) only connect as hands-free call devices (HFP/SCO) and cannot stream media. Arcane executes:

```
audioManager.startBluetoothSco()  
audioManager.isBluetoothScoOn = true
```
- 4. Silent PCM Keep-Alive Track:** If an app opens but does not instantly open an audio recording line, Android's audio server tears down the SCO link within 500ms. To prevent this, Arcane starts a lightweight 8000Hz 16-bit mono `AudioTrack` with `USAGE_VOICE_COMMUNICATION` streaming silence. This pins the SCO link open until the assistant binds to the mic.
- 5. 120s Safety Watchdog:** A dedicated `Handler.postDelayed` runnable automatically calls `stopAudioRouting()` after 120 seconds, preventing the phone from getting trapped in permanent phone-call mode.

BLUETOOTH MIC CAPTURE & AUTO-TAP ARCHITECTURE (CONT.)

PAGE 2
OF 2

B. Mic Tap Sync Latency Calibration (`LauncherTakeoverService.kt`)

Smartwatches and Bluetooth headsets require **800ms to 1500ms** to establish the SCO audio link and negotiate audio codecs. If the assistant's mic button is tapped immediately upon window launch, the assistant grabs the phone's internal microphone instead of the watch!

Latency Solution: Arcane delays the auto-tap execution by a user-calibrated interval (configurable between `200ms` and `4000ms`, defaulting to `1000ms`).

```
// LauncherTakeoverService.kt
val delayMs = getMicClickDelay(this)
Log.i(TAG, "Waiting $delayMs ms for watch/Bluetooth mic ready before auto-tap...")
mainHandler.postDelayed({
    executeRecordedTap(pendingAuto)
}, delayMs)
```

C. Toggle Protection via Active Recording Inspection

In third-party AI assistants, the voice/mic button is a toggle switch (tap to start, tap again to cancel/hang up). If the operator taps their watch assistant button while already in a voice session, a naive auto-tap would terminate the conversation!

Arcane checks active hardware recording configurations before dispatching taps:

```
// LauncherTakeoverService.kt
private fun sessionLikelyActive(context: Context, pkg: String): Boolean {
    val last = lastAutoTapAt[pkg] ?: return false
    if (SystemClock.elapsedRealtime() - last >= AUTO_TAP_SESSION_WINDOW_MS) return false
    val am = context.getSystemService(Context.AUDIO_SERVICE) as? AudioManager ?: return false
    return try { am.activeRecordingConfigurations.isNotEmpty() } catch (_: Exception) { false }
}
```

Because Arcane itself is not recording when the assistant intent arrives, any active recording configuration belongs to the external AI assistant. If an active session is detected, the auto-tap is gracefully bypassed.

D. Full End-to-End Execution Sequence

- 1 Hardware Trigger Ingestion:** Smartwatch or Bluetooth button triggers `ACTION_VOICE_COMMAND` or `ACTION_ASSIST`. `MainActivity.kt` catches the intent.
- 2 Lock Screen Dismissal (Movement 1):** If device is locked, Arcane wakes the screen and executes the calibrated unlock swipe gesture (`executeUnlockSequence`).

- 3 **Audio SCO Bridge Engaged:** `BluetoothAudioRouter.routeAudio(enable=true)` forces communication mode, claims Bluetooth SCO device, and starts silent PCM keep-alive track.
- 4 **Assistant Launch & Arming:** Target app (e.g. ChatGPT, Gemini) is launched via `launchAssistantVoiceMode`. `LauncherTakeoverService.armAutoTap` is primed.
- 5 **Delayed Precision Tap (Movement 2):** Upon window transition, the service waits for the configured mic delay (e.g. 1000ms), verifies session is not already active, and dispatches direct touch tap gesture `dispatchGesture()` at calibrated `(xRatio, yRatio)`.